

check how well your ML model is performing
(generalize well?)

Week 3, Model evaluation and selection

CAPP 30254, Machine Learning for Public Policy

choose the best ML model for
your problem/dataset

bias-variance tradeoff

CAPP 30254

MACHINE LEARNING FOR PUBLIC POLICY

— SPRING 2025 —

What's next?

New vocabulary:

evaluation: How well a trained model performs

validation: How well a model performs during training

* avoid overfitting *

We started the quarter by learning about the fundamental of machine learning (vocabulary, notation, etc.).

feature

x_i y_i $\hat{y}_i$ x

Then we saw various supervised classification models:

■ Least squares

classification/
regression

learn w by min. L

■ Ridge regression

$$\hat{y} = \begin{cases} +1 & w^T x \geq 0 \\ -1 & < 0 \end{cases}$$

$$\hat{y} = w^T x$$

■ Perceptron

classification

■ Logistic regression

probability

learn w

$$\hat{y} = \begin{cases} 1 & w^T x \geq \text{prob } p \\ 0 & 1-p \end{cases}$$

and related topics like gradient descent.

SGD

to find w

Now we are going to switch from learning about specific models to learning about model evaluation and validation. How do we determine whether our model is good? How do we pick between different potential models?

How do we know a model works?

important!

Once we train a model, how do we know it works? *all types of input*

In 2015, Ian Goodfellow et al., published the paper "Explaining and Harnessing Adversarial Examples". In the paper, the team fed a picture of a panda into an image classification model. The model correctly classified the animal as a panda with a 57% *high* confidence.



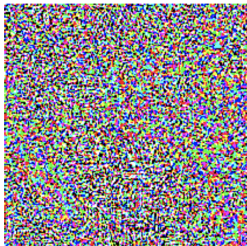
How do we know a model works?

models:
fragile
overconfident

Then they added a small amount of noise to the picture. The noise was imperceptible to the human eye but was designed to confuse the model.

} adversarial

noise / perturbation



looks the same



small change in
input → big change
in output

Bad for:

- * reliability
- * robustness
- * attacks

What do you think the model predicted now?

Predicted "gibbon" with 77.3%

Model evaluation on training data

Once we train a model, how do we know it works?

When we evaluate a model we assess the model's performance on unseen data and make sure it will work well in the real world.
↳ generalizes well

We can use the model's performance on the training data to estimate its overall performance. We compute the average *training error* (the error from the training data) using the loss function of the model:

$$\text{training error} = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i)$$

average loss

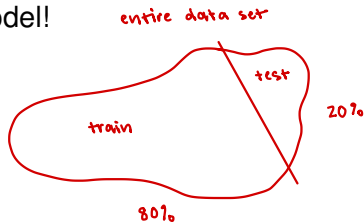
Test set

training error doesn't tell us about
the generalization error

Using the training data to evaluate a model can be misleading. The training error tells us how well the model performs *on the data it was trained on*.

A better way to evaluate a model is to use a *test set* that is separate from our training data. We'll use the training data to train a model, then use the test set to evaluate the model after it has been trained.

Never use your test set to train your model!



Validation set



Another set called the *validation set* should be used to train the model's hyperparameters such as strength term λ in ridge regression or the step size η in gradient descent.

To summarize:

- Training set: data used to train the model w
- Validation set: data used to tune the model and prevent overfitting m, λ, η
- Test set: data used to evaluate the model after the model has been fully trained

model parameters/hyperparameters: Next time

polynomial boundary degree m

ridge regression:

$$\lambda \leq (w_i)^2$$

gradient descent:

$$w^{t+1} = w^t - \eta \nabla_w \ell(w^t)$$

w
DS 1

Cross validation

k-fold cross validation

why?

more reliable estimate
of performance

catches under/overfitting

computationally expensive
can be slow

Cross validation is widely used to evaluate the performance of machine learning models.

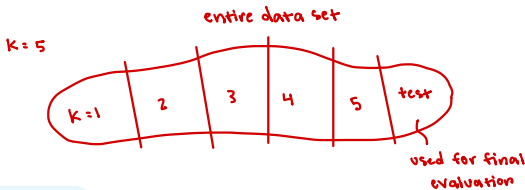
Cross validation works by splitting your dataset into k “folds” of data. Use one fold as the validation set while the other $k - 1$ folds are used to train your model.

wasteful to use validation set just once

So, you will train your model k times and use the average performance across all k runs.

Pick k :

$k = 5, 10$



validation fold	training folds
1	2-5
2	1, 3-5
3	1-2, 4-5
4	1-3, 5
5	1-4

then average the results

Cross validation

Cross validation is widely used to evaluate the performance of machine learning models.

Cross validation works by splitting your dataset into k “folds” of data. Use one fold as the validation set while the other $k - 1$ folds are used to train your model.

So, you will train your model k times and use the average performance across all k runs.

Cross validation for polynomial selection:

select the best model

for each m : $m = 1, 2, \dots, 6$

split data into K sets

train the model k times error for each: E_m^i $i = 1, \dots, k$

estimate the error E_m average = $\frac{1}{k} \sum E_m^i = \hat{E}_m$

select the best model

$$m = \underset{m}{\operatorname{argmin}} \hat{E}_m$$

pick lowest validation error

Accuracy

For many classification tasks, we can use the *misclassification error rate*

$$\text{error rate} = \frac{1}{n} \sum_{i=1}^n I(\hat{y}_i \neq y_i)$$

$I(\hat{y}_i = y_i) = 0$
 $I(\hat{y}_i \neq y_i) = 1$

and *accuracy*:

$$\text{accuracy} = \frac{1}{n} \sum_{i=1}^n I(\hat{y}_i = y_i) = 1 - \text{error rate}$$

counts accurate predictions

to evaluate our models.

$$\frac{TP + TN}{TP + FP + TN + FN}$$

Confusion matrix

The *confusion matrix* for a binary classification problem is a table that counts the number of true and false positives and true and false negatives produced by a model.

	predicted labels	
	$\hat{y}_i = +1$	$\hat{y}_i = -1$
actual/ true labels	$y_i = +1$ $\sum_i 1$ when $y_i = \hat{y}_i$	$y_i = +1$ $\sum_i 1$ when $y_i \neq \hat{y}_i$
	$y_i = -1$ $\sum_i 1$ when $y_i \neq \hat{y}_i$	$y_i = -1$ $\sum_i 1$ when $y_i = \hat{y}_i$

Ex. spam

true positive TP	false negative FN
false positive FP	true negative TN

Precision, recall, F1

Ex. logistic classifier for people eligible for an emergency subsidy based on socioeconomic factors.

We're willing to accept a large FP in order to secure a high $\frac{\text{TP}}{\text{recall}}$. Should we set the threshold of our classifier high/low?

Precision measures how accurate the positive predictions are.

$$\frac{\text{TP}}{\text{TP} + \text{FP}}$$

of + predictions, how many were correct

Spam: FN cost less than FP

+1 if $P \geq \text{th}$
-1 otherwise
missing important email

Recall measures the fraction of positive labels that are correct.

$$\frac{\text{TP}}{\text{TP} + \text{FN}}$$

of actual +, how many did we find?

covid test: FP cost less than FN

missing sick case

The *F1 score* is a balance between precision and recall.

$$\frac{2 \cdot \text{precision} \cdot \text{recall}}{\text{precision} + \text{recall}}$$

Regression evaluation:

MAE

MSE



2

²Generated by GPT-4, DALL-E 3 after the prompt: “Hi, could you please make a minimalist logo for a machine learning class...”